Empirical Software Engineering (2026) 31:27
https://doi.org/10.1007/s10664-025-10777-0

SSFuzz: Synthesizing and scheduling bug-triggering code segments for history-driven compiler testing

Tianmin Hu[1] · Zhenye Fan[1] · Zhanbo Ye[1] · Guixin Ye[1] · Lu He[1]

Received: 4 March 2025 / Accepted: 11 November 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract
History-driven fuzzing is a viable method to expose compiler bugs. It leverages historical information, such as historical test programs and historical execution information, to locate error-prone modules and conduct continuous testing. However, existing works are inefficient in utilizing historical information, resulting in a limited bug-triggering capability. In this work, we propose SSFuzz, a compiler fuzzer that synthesizes bug-triggering code segments into test cases and schedules both seed programs and code segments. SSFuzz collects bug-triggering code segments from bug-related pull requests and synthesizes them into test cases using different strategies. During the period of testing, SSFuzz uses an entropy-based scheduler to prioritize code segments based on the behavior of target JVMs, and retain test cases as seed programs according to the anomalous behavior feedback and code coverage feedback. During the bug submission phase, SSFuzz successfully uncovered 20 previously undetected bugs across all three target JVMs. Out of these, 14 have been verified, and 3 have already been fixed. Compared with state-of-the-art fuzzers, SSFuzz uncovers $1.3 \sim 3.1\times$ more bugs and achieves the highest code coverage.

Keywords Software testing · Compiler testing · Bug finding · JVM

Tianmin Hu and Zhenye Fan contributed equally to this work.

Communicated by: Lei Ma, Roberto Pietrantuono.

This article belongs to the Topical Collection: *Special Issue on Software Reliability Engineering*.

✉ Guixin Ye
gxye@nwu.edu.cn

Tianmin Hu
hutianmin@stumail.nwu.edu.cn

Zhenye Fan
fanzhenye@stumail.nwu.edu.cn

Zhanbo Ye
cain@stumail.nwu.edu.cn

Lu He
helu@nwu.edu.cn

[1] Northwest University, Xi'an, China

Published online: 05 December 2025

1 Introduction

The Java Virtual Machine (JVM) is an abstract computing engine that allows Java programs to run on any device or operating system. The Java compiler (javac) translates Java source code into bytecode, which is then executed by the JVM. Several JVM implementations are available, including HotSpot (Oracle 2025) and GraalVM (Oracle 2025) from Oracle, OpenJ9 (IBM 2025) and JikesRVM (IBM 2025) from IBM, as well as Zulu (Azulzulu 2025) and Zing (Azuzing 2025) from Azul. Just like any other complex software, although these JVMs are meticulously maintained by professional teams, these implementations remain susceptible to bugs and security vulnerabilities (Yang et al. 2011; Chen et al. 2020).

Fuzzing has proven to be an effective method for uncovering compiler bugs (Chen et al. 2020). Moreover, recent advancements have integrated history-driven techniques into compiler fuzzers. These methods leverage historical information, such as historical test programs and execution information, to improve their effectiveness in exposing bugs.

To locate error-prone modules in JVMs rapidly, several studies (Holler et al. 2012; Zhao et al. 2022; Gao et al. 2023; Zhong 2022) leverage historical test programs to generate high-quality test cases. JavaTailor (Zhao et al. 2022) extracts code segments from historical bug-triggering test cases and synthesizes new test cases by randomly inserting these segments into seed programs. However, many extracted segments contain redundant semantics, resulting in a large number of test cases that fail to trigger new bugs. VECT (Gao et al. 2023) addresses this issue by clustering semantically similar code segments and selecting clusters based on the quality of the generated test cases. Nevertheless, many clusters fail to contribute to triggering new bugs.

Furthermore, to fix the broken dependencies of the extracted code segments, both methods employ two strategies: reusing variables in the seed program and constructing new definitions. Although these resolutions can significantly improve the syntax correctness of the synthesized test cases, they can not increase the bug-triggering capability of the test cases. Therefore, *how to collect bug-triggering code segments and assemble them into the seed programs* is still a challenge in JVM fuzzing.

To ensure the continuous detection of new bugs, existing methods (Gao et al. 2023; Wu et al. 2023; Chen et al. 2016, 2019; Ye et al. 2023) integrate scheduling schemes into fuzzers. After each testing round, these fuzzers utilize historical execution information to guide subsequent test iterations. For example, SJFuzz (Wu et al. 2023) designs a seed scheduler based on diversity and discrepancy guidance and selects mutants by maximizing program diversity. VECT (Gao et al. 2023) selects clusters according to two criteria: bug-triggering capability and diversity. However, these methods do not incorporate a deep understanding of the behavior of JVM implementations during testing, leading to a convergence state of code coverage and limited bug-triggering capability. Therefore, *how to leverage execution information during JVM fuzzing* is another unsolved challenge.

In this work, we introduce SSFuzz,[1] a compiler fuzzer that synthesizes bug-triggering code segments into test cases and schedules both seed programs and code segments. To enhance the collection of high-quality bug-triggering code elements, SSFuzz extracts code segments and variable declarations from bug-related Pull Requests (PRs) in GitHub repositories of target JVMs. Then, SSFuzz employs three strategies to fix the broken dependencies of the code segments when assembling test cases, including (1) searching in the seed programs, (2) search-

[1] **S**ythesizing and **S**cheduling bug-triggering code segments for compiler **Fuzz**ing

ing in the variable declaration database, and (3) prompting a large language model (LLM). To uncover novel behaviors in target JVMs, we integrate an entropy-based scheduler that prioritizes high-entropy code segments. Test cases are retained as seed programs based on the bug-triggering capability of the test cases and the code coverage of the JVMs.

To evaluate the effectiveness of SSFuzz, we conducted an extensive experiment on three popular JVMs, including HotSpot, OpenJ9, and GraalVM. During the bug submitting period, SSFuzz exposed 20 previously unknown bugs in all three target JVMs, 14 of them have been confirmed, and 3 have been fixed. The comparison experiment results have shown that SSFuzz outperforms four SOTA fuzzers (Zhao et al. 2022; Gao et al. 2023; Wu et al. 2023; Zhang et al. 2024). SSFuzz detected $1.3 \sim 3.1\times$ more bugs and achieve the highest code coverage.

To sum up, our work makes the following contributions:

- We propose SSFuzz, which locates error-prone modules and conducts continuous testing on target JVMs by synthesizing and scheduling bug-triggering code segments.
- We conduct a wide evaluation of three popular JVM implementations, demonstrating the effectiveness of SSFuzz. In particular, SSFuzz detects 20 previously unknown bugs, 14 of which have been confirmed.
- We implemented the prototype of SSFuzz and released it on GitHub (https://github.com/ssfuzz/ssfuzz) for replication and extension.

2 Background and Motivation

2.1 Historical Information in Compiler Fuzzing

One of the historical information in compiler fuzzing is historical test programs. The insight is that code segments extracted from historical test programs can be utilized to generate test cases that can cover more error-prone code branches, leading to the detection of new bugs. One of the classical techniques is LangFuzz (Holler et al. 2012). It extracts code segments from the test suite of JS engines and replaces partial code in the seed program with the extracted code segments. More recently, LERE (Zhong 2022) classifies elements of bug reports into three categories: programs, comments, and others. It then serves programs from bug reports as test cases for compiler testing. JavaTailor (Zhao et al. 2022) extracts code segments from historically bug-triggering test programs and the test suite of JVMs and inserts them into seed programs.

However, such a set of historical test programs has been accumulated from the initial development phase to the present. Due to ongoing feature development and bug fixes, only partial programs remain relevant to the latest JVM versions. Consequently, collecting all code segments from this source is ineffective for detecting bugs in the most recent JVM releases.

In contrast to existing approaches, SSFuzz further narrows the source of code segments, effectively avoiding the extraction of segments with low bug-triggering potential. Additionally, SSFuzz designs two new strategies for test case synthesis to resolve dependencies within the extracted code segments more accurately and provide greater coverage of edge cases.

Another historical information is historical execution information. Some compiler fuzzers utilize information in the current round to guide testing in the next iteration. Both

ClassFuzz (Chen et al. 2016) and ClassMing (Chen et al. 2019) adopt the Metropolis-Hastings algorithm (Huang et al. 2017) to guide their mutation processes. ClassFuzz utilizes the algorithm to prioritize its mutators, while ClassMing utilizes the algorithm to effectively produce diverse mutants. ComFuzz (Ye et al. 2023) activates bug-guided or general-purpose mutators based on criteria for anomalous behavior and code coverage. SJFuzz (Wu et al. 2023) introduces a mutation scheduling strategy based on the Monte Carlo method, which selects the optimal mutator by computing the diversity between the original seeds and the mutated class files, while incorporating random mutations to avoid local optima. VECT (Gao et al. 2023) employs a classification-based aggregation scheduling strategy for synthesized components, optimizing their selection by prioritizing groups that are more likely to trigger bugs, while also maintaining code snippet diversity.

Despite recent advancements in these methods, fine-grained metrics offer greater potential for identifying unique bugs (Wang et al. 2021). To this end, we introduce an entropy-based code segment scheduler that computes the Shannon entropy of JVM coverage. By leveraging entropy as a continuous signal, SSFuzz avoids overemphasizing frequently visited paths and guides the fuzzing process toward less explored execution regions, thereby improving the possibility of uncovering previously undetected bugs.

2.2 Motivation Example

Listing 1 exposes a functional bug in OpenJ9. For brevity, we have removed redundant statements from the original test case. When the variable `methodHandle` is assigned as `null` and the method `invokeExact` is called, the JIT compiler of OpenJ9 crashes with the message "Unable to locate JIT stack map - aborting VM". This occurs because the JIT attempts to load the `type` field of a `null` value, leading to an unexpected segmentation fault. After we reported the issue, the developers confirmed and promptly addressed the bug.

```
1  import java.lang.invoke.MethodHandle;
2
3  public class MyJVMTest {
4    void performTest() {
5      int innerCounter = 1;
6      for (int index = −3230; index < 9840; index += 2) {
7        // LLM−generated try−catch statement (lines 8−15)
8        try {
9          // LLM−generated declaration (line 10)
10         MethodHandle methodHandle = null;
11         // Bug−triggering code segment (line 12)
12         methodHandle.invokeExact();
13       } catch (Throwable throwable) {
14       } finally {
15       }
16     }
17   }
18
19   public static void main(String[] args) {
20     MyJVMTest instance = new MyJVMTest();
21     instance.performTest();
22   }
23 }
```

Listing 1 A test case generated by SSFuzzdetects a functional bug of OpenJ9

The bug-triggering code segment, shown in line 12, is extracted from Pr#13855 (OpenJDK 2023). It was selected due to its high priority, as determined by the code segment scheduler. During synthesis into the seed program, the LLM generates the initialization statement for the variable `methodHandle` and wraps the code segment in a try-catch block. Additionally, the large loop statement in the seed program successfully activates the JIT compiler of OpenJ9.

These observations lead to several key insights: *(1) Code segments extracted from test programs in bug-related PRs can effectively trigger new bugs. (2) Novel strategies, such as prompting an LLM, can resolve broken dependencies in code segments and increase the probability of triggering bugs. (3) Carefully designed scheduling mechanisms can optimize selection processes, thereby enhancing the efficiency of bug detection.*

3 Design

Figure 1 depicts the overview of SSFuzz. Overall, SSFuzz collects test programs from bug-related PRs and extracts variable declarations and bug-triggering code segments from these test programs (Section 3.1). Then, code segments are inserted into seed programs to synthesize test cases (Section 3.2). During differential testing, test cases are executed on several target JVMs (Section 3.3). To ensure continuous and effective testing, SSFuzz schedules seed programs and code segments based on anomalous behavior feedback and code coverage feedback (Section 3.4).

3.1 Bug-Triggering Code Element Collection

To maximize the utility of historical test programs, SSFuzz collects both variable declarations and bug-triggering code segments.

Test Program Collection First, we implement an HTML crawler to collect all the PRs from the GitHub repositories of the test JVMs. Then, we retain the test programs from bug-

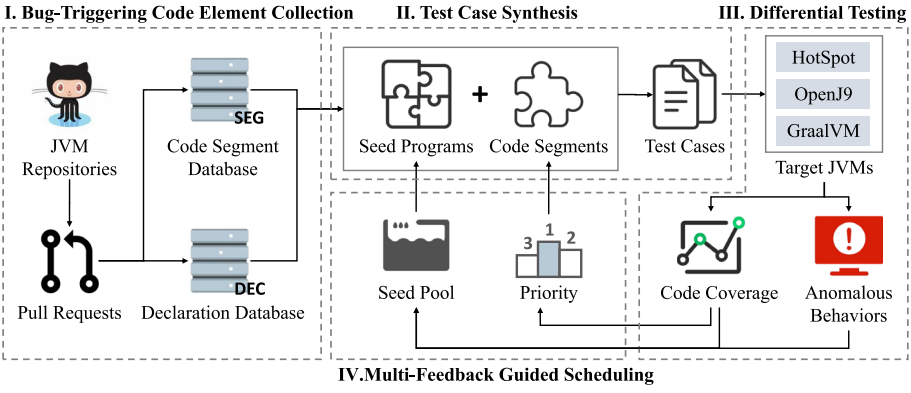


Fig. 1 The overview of SSFuzz

related PRs. Specifically, we identify PRs labeled as "bug" and store Java files whose names include the keyword "test".

Code Segment Extraction After that, we use JavaParser (Hosseini and Brusilovsky 2013), a parser for Java programs, to parse the test programs and extract statement-level code segments from ASTs. Inspired by previous works (Liu et al. 2020; Zhu and Böhme 2021), we can find that not all code segments are equally important. Some modules are no longer frequently updated or are less error-prone. To address this, we focus on extracting segments that are more likely to trigger bugs in recent JVMs. Specifically, we solely focus on newly-added code segments, i.e., those containing added codes of PRs, and store them in the code segment database $\mathcal{C}$. To ensure that the code segment can be seamlessly integrated into a seed program without causing syntax errors, we need to import the required packages and user-defined methods. Therefore, we also record the import statements and user-defined methods for each segment in $\mathcal{C}$.

Variable Declaration Extraction Some test programs may include self-defined data types that are challenging for fuzzers to initialize. If code segments are extracted without their corresponding declaration statements, the resulting test cases may fail to execute. Moreover, variable declaration statements in test programs often cover edge cases. Therefore, we use JavaParser to extract all variable declaration statements in the test programs and store them in the variable declaration database $\mathcal{V}$. Similar to $\mathcal{C}$, we also record import statements and user-defined methods for each variable declaration. As a result, we collected 7K test programs and extracted 90K code segments and 100K variable declarations.

Example Listing 2 displays partial code of Pr#17699 (Eclipse-OpenJ9 2023). The return statement (lines 212-213) is one of the added codes in PR#17699. Therefore, it is extracted as a bug-triggering code segment and stored in $\mathcal{C}$.

```
210  public final List<String> getInputArguments() {
211    checkMonitorPermission();
212    return Collections.unmodifiableList(
213      ManagementUtils.convertStringArrayToList(VM.getVMArgs()));
214  }
```

Listing 2 Partial code of Pr#17699 (n.d.).

3.2 Test Case Synthesis

To obtain diverse seed programs, SSFuzz constructs an initial seed pool by using the generator of previous work (Ye et al. 2023). During the testing period, SSFuzz selects a seed program from the seed pool and searches for a code segment from $\mathcal{C}$. Inserting a code segment into a seed program without creating interaction is of limited value; therefore, the selected code segment must share at least one variable of the same type with the seed program.

Since the context of the extracted code segments is incomplete, there may exist some undeclared variables. To synthesize valid test cases, SSFuzz employs three strategies to fix these missing dependencies: (1) searching for variables of the same type in the seed pro-

gram, (2) searching the variable declaration database $\mathcal{V}$ for matching variable types, and (3) prompting an LLM to generate initialization statements.

Searching in the Seed Programs For an undeclared variable, SSFuzz prioritizes identifying variables declared in the seed program. This approach enhances the interaction between the code segment and the seed program. Specifically, SSFuzz utilizes JavaParser to parse the seed program and identifies `FieldDeclaration` nodes in the AST. It then searches the resulting list for variables with matching types.

Searching in variable declaration database $\mathcal{V}$ If no suitable variables are found in the seed program, SSFuzz proceeds to search the variable declaration database $\mathcal{V}$, which contains a diverse set of variables with complex types. Since all relevant information is stored during extraction, SSFuzz can efficiently query the database by matching the variable type.

Prompting an LLM Due to the complex interactions among variables and methods in Java programs, it is not always possible to find proper variable declarations from the seed program and the variable declaration database. Therefore, we utilize the chain of thought (CoT) approach to generate declaration statements when appropriate ones are absent in both the seed programs and database $\mathcal{V}$. This strategy is designed for two main reasons: (1) Datasets of LLMs contain various test information, enabling the models to identify values that can trigger compiler bugs. (2) LLMs can refine test cases by correcting syntax errors and enhancing the interaction between code segments and seed programs. The prompt template is shown in Fig. 2. The template begins with the context and primary requirement in the first paragraph, followed by the synthesized test case and the selected code segment. For each

Prompt Template

Role: Senior JVM Testing Engineer
Task: Be sure to give me the modified test case with "'java "' qualifiers !
Expertise Requirements: (1)10+ years Java/JVM testing experience, (2) Deep understanding of JVM internals (ClassLoader, JIT, GC, etc.), (3) Expert in fuzzing test generation, (4)Proficient in vulnerability pattern recognition.
Task Objective: Generate executable Java test cases that effectively trigger JVM defects by synthesizing seed inputs and vulnerability code snippets. Prioritize coverage of these modules: Memory Management, Class Loading, JIT Compilation, Thread Scheduling.

The following is the content of a Java test case synthesized from a certain seed program and a certain code segment:
{test_case}
The above test case was synthesized from the following bug-triggering code segments:
{code_segment}

Please analyze the test cases based on the above and complete the following tasks (Think step-by-step):
1. Determine whether there is any variable type and value mismatch, please solve the problem directly.
2. Determine whether there are any variable conflicts during synthesis, please solve the problem directly and properly.
3. Whether there are undefined variables or classes, etc., directly make up for these missing context variables, and make sure that you can directly execute this test case.
4. You must ensure that the defect-triggering code fragment can interact with the seed, otherwise there is no difference between executing two test cases.
5. Directly after the organization of the test case output, and in the resulting Java program to add a line of comments to show what you have done.

Fig. 2 The prompt template used in SSFuzz

test case, two placeholders are filled with the current test case and the corresponding code segments. The final section outlines step-by-step tasks to fulfill the primary requirement.

Algorithm 1 Test case synthesis.

Inputs: *seed*:a seed program, *segment*:a bug-triggering code segment
Outputs: *testcase*: a synthesized test case
1: $\mathcal{D} \leftarrow GetDeclaredVariables(seed)$
2: $\mathcal{U} \leftarrow GetUnDeclaredVariables(segment)$
3: $declarations \leftarrow []$ /* Store the newly generated declarations */
4: $needLLM \leftarrow False$ /* Record if this testcase need the third synthesis strategy */
5: **for** *var* in $\mathcal{U}$ **do**
6: /* Situation 1: If a matching type exists in the declaration of *seed* */
7: $available_declarations \leftarrow SearchFromSeed(var, \mathcal{D})$
8: **if** $available_declarations.length > 0$ **then**
9: $declarations.append(SelectAndAssign(available_declarations))$
10: continue
11: **end if**
12: /* Situation 2: If available declarations in the variable database */
13: $available_declarations \leftarrow SearchFromDatabase(var)$
14: **if** $available_declarations.length > 0$ **then**
15: $declarations.append(SelectAndAssign(available_declarations))$
16: continue
17: **end if**
18: /* Situation 3: When the above two conditions are not met */
19: $needLLM \leftarrow True$
20: **end for**
21: $testcase \leftarrow Assemble(seed, segment, declarations)$
22: **if** $needLLM$ **then**
23: $testcase \leftarrow GenerateWithLLM(testcase)$
24: **end if**
25: **return** *testcase*

The detailed process of test case synthesis is described in Algorithm 1. Given a seed program *seed* and a code segment *segment*, SSFuzz first derives declared variables $\mathcal{D}$ in the seed program *seed* and undeclared variables $\mathcal{U}$ in the bug-triggering code segment *segment* (lines 1-2). For each undeclared variable *var* in *segment* (line 5), SSFuzz checks whether there are variables of the same type in *seed* (line 7). If matching variables are found, SSFuzz randomly selects one and assigns it to *var* (lines 8-11). If no matching variables are present, SSFuzz searches the variable declaration database $\mathcal{V}$ (line 13). If suitable variables are found, SSFuzz randomly selects one and assigns it to *var* (lines 14-17). If no variable is found through these methods, a flag *needLLM* is set to *true* (line 19). Upon completing the loop, SSFuzz combines *declarations*, *segment*, and *seed* into a new test case (line 21). If *needLLM* is *true*, this test case is then sent to the LLM to fix the dependencies (lines 22-24). Finally, variable *testcase* is returned (line 25).

Several positions may be available for inserting the code segment. SSFuzz randomly selects one of these positions within the Java methods of the seed program, excluding the first and last positions. This design ensures that the generated test cases execute the code segments in conjunction with other parts of the program and also helps maintain the diversity of the generated test cases.

3.3 Differential Testing

During differential testing, SSFuzz executes test cases across various JVMs, including HotSpot, OpenJ9, and GraalVM. Similar to previous compiler fuzzers (Zhao et al. 2022; Ye et al. 2023, 2021), SSFuzz identifies three types of anomalous behaviors: inconsistency, timeout, and crash. Inconsistency occurs when the output or behavior of a test case varies across JVMs. A timeout is detected when a test case exceeds the expected execution time or causes the JVM to hang. The timeout threshold is set to 60 seconds by default. Crash refers to critical errors that occur during test case execution.

Furthermore, to reduce the cost of manual analysis, SSFuzz utilizes several rules to filter out analyzed anomalies. Specifically, SSFuzz checks whether specific keywords exist in test cases or outputs, including those related to threads, time, and random operations. For instance, inconsistencies arising from different JVMs producing varying thread numbers are considered false positives. SSFuzz will verify whether all outputs include the phrase "Thread [A NUMBER]"; if this condition is met, the anomalous behavior is disregarded.

3.4 Multi-Feedback Guided Scheduling

Although the search space of code segments is reduced by extracting bug-triggering ones, a random selection of code segments remains an inefficient strategy. Additionally, a fixed seed pool limits the diversity of the generated test cases. To address these issues, SSFuzz employs two schedulers; one is for storing novel seed programs and another for prioritizing bug-triggering code segments.

3.4.1 Seed Scheduling

We adopt two feedback mechanisms for seed scheduling: anomalous behavior feedback and code coverage feedback. Since test cases that trigger bugs have already covered fragile code branches, generating new test cases from these cases increases the likelihood of uncovering additional bugs. Therefore, any test case that triggers anomalous behavior is added to the seed pool for further synthesis. While this may lead to repeated detection of the same bugs, we mitigate redundant effort by the automated analysis script introduced in Section 3.3.

Following the approach of AFL-series studies, we also retain test cases that achieve broader code coverage in JVMs. By reusing these high-coverage test cases, SSFuzz can continuously explore previously untested execution paths.

3.4.2 Code Segment Scheduling

As demonstrated in previous work (Böhme et al. 2020), Shannon's entropy (Shannon 1948) can be used to measure how efficiently a fuzzer discovers new behaviors. Building on this idea, SSFuzz computes the entropy of code coverage in JVMs and uses it to guide the prioritization of code segments. Unlike raw coverage, which only records whether a path is exercised, entropy reflects the frequency distribution of covered paths. As a result, entropy can increase even when total coverage remains constant, for example, when test cases trigger particularly rare paths. Specifically, SSFuzz executes the test case and obtains the code coverage of a target JVM. Let p_i represent the coverage frequency of each line or branch. Since

some lines or branches may execute multiple times in loop statements, where $\sum_{i=1}^{n} p_i > 1$, p_i must be normalized as follows:

$$p_i' = \frac{p_i}{\sum_{j=1}^{n} p_j} \tag{1}$$

Then, the information entropy of code coverage is calculated as:

$$\begin{aligned} H(segment) &= -\sum_{i=1}^{C} p_i' \log(p_i') \\ &= \log\left(\sum_{j=1}^{C} p_j\right) - \frac{\sum_{i=1}^{C} p_i \log(p_i)}{\sum_{j=1}^{C} p_j} \end{aligned} \tag{2}$$

Where C represents the total number of code lines and branches. A higher information entropy indicates that the corresponding code segment in the test case has a greater probability of being selected for further testing. Notably, due to the minimal difference in function coverage, we solely include line coverage and branch coverage from the calculation of information entropy.

4 Experimental Setup

4.1 Research Questions

In the evaluation, we aim to address the following research questions:

- **RQ1:** How does SSFuzz perform in detecting JVM bugs?
- **RQ2:** Can SSFuzz outperform existing baselines?
- **RQ3:** How effective is the synthesis and scheduling module of SSFuzz?
- **RQ4:** How do different LLMs perform in test case synthesis?

4.2 Target Compilers

In our experiments, we selected the latest release versions of JVMs from various vendors, including HotSpot, OpenJ9, and GraalVM, as the test subjects. The specific versions are listed in Table 1. For all comparison experiments, we used HotSpot as the baseline JVM to evaluate code coverage. This choice was made because prior studies (Zhao et al. 2022;

Table 1 Target JVMs tested in our evaluation

JVMs	JDK Version	Build Number	Release Date
HotSpot	17	17.0.11+9	Apr. 2024
	11	11.0.23+9	Apr. 2024
OpenJ9	17	17.0.9+8	Apr. 2024
	11	11.0.17+8	Apr. 2024
GraalVm	17	17.0.9+9	Oct. 2023
	11	11.0.20+8	Jul. 2023

Gao et al. 2023; Wu et al. 2023; Ye et al. 2023) have also employed HotSpot for evaluating the code coverage of their proposed methods. Additionally, HotSpot is the most widely used JVM among the three tested JVMs.

4.3 Competitive Baselines

We compare SSFuzz with four state-of-the-art fuzzers. Two of these fuzzers leverage historical test programs to generate test cases (Zhao et al. 2022; Gao et al. 2023), while the other two introduce novel scheduling schemes (Wu et al. 2023; Zhang et al. 2024).

- JavaTailor (Zhao et al. 2022). A JVM fuzzer that extracts code segments from historical test programs and randomly inserts them into seed programs to generate new test cases.
- VECT (Gao et al. 2023). A JVM fuzzer that clusters semantically similar code segments and employs a feedback-driven strategy for code segment selection.
- SJFuzz (Wu et al. 2023). A JVM fuzzer that employs seed and mutator scheduling mechanisms for automated JVM differential testing.
- MobFuzz (Zhang et al. 2024). A gray-box fuzzer that enhances the performance of coverage-guided fuzzing by simultaneously optimizing multiple objectives. Note that MobFuzz does not provide accessible artifacts and is not designed for JVM fuzzing. Therefore, we carefully replicate its scheduler based on the description in the paper and substitute the original scheduler in SSFuzz with it. To distinguish our reimplementation from the original tool, we refer to it as MobFuzz*.

4.4 Metrics

We evaluated SSFuzz via three key metrics in our experiments:

Bug-triggering Capability Following prior work on JVM Fuzzing (Zhao et al. 2022; Gao et al. 2023; Wu et al. 2023; Ye et al. 2023), we report the number of detected anomalies.

Syntax Passing Rate We calculate the ratio of test cases that successfully compile to the total number of test cases.

Code Coverage We use GCOV (GNU 2025) and LCOV (Hinds et al. 2014) to instrument .cpp files in HotSpot and obtain code coverage. Three types of code coverage are counted, including line coverage, branch coverage, and function coverage.

4.5 Evaluation System

We implemented SSFuzz in around 6k lines of Python code and 12k lines of Java code. We selected GPT-4-turbo for test case synthesis because it yielded the best overall experimental results (see Section 5.5). We conducted experiments on a high-performance server equipped with an Intel i7-14700K processor (3.40 GHz) and 48 GB of RAM. The version of the operating system is Ubuntu 22.04.

5 Results and Analysis

5.1 Detected Bugs (RQ1)

Table 2 shows the previously unknown bugs detected by SSFuzz. At the time of submission, SSFuzz discovered a total of 20 previously unknown bugs, of which 14 were confirmed and 3 were fixed.

```
1  import sun.misc.Unsafe;
2  import java.lang.reflect.Field;
3  import java.nio.*;
4  import sun.nio.ch.DirectBuffer;
5
6  public class MyJVMTest {
7    void getAll() throws Exception {
8      Field theUnsafe = Unsafe.class.getDeclaredField("theUnsafe");
9      theUnsafe.setAccessible(true);
10     // Code segment synthesized in first iteration (line 11)
11     Unsafe unsafe = (Unsafe)theUnsafe.get(null);
12     // Fixed Dependencies of second code segment (lines 13—16)
13     ByteBuffer b = ByteBuffer.allocateDirect(0);
14     long addr = ((DirectBuffer)b).address();
15     int value = 10;
16     int len = (int)Math.pow(2,7);
17     // Code segment synthesized in second iteration (lines 18—20)
18     for (int i = 0; i < len; i++){
19       unsafe.putByte(null,addr+i,(byte)value);
20     }
21   }
22
23   public static void main(String[] args) throws Exception {
24     System.out.println(new MyJVMTest().getAll());
25   }
26 }
```

Listing 3 A test case that triggers a bug in OpenJ9

Bug Example The test case in Listing 3 exposes a functional bug of OpenJ9. In line 19, the method putByte attempts to write the byte `0x0A` (decimal 10) to the memory address `addr + i`. However, OpenJ9 does not implement optimization for the write operation with the `Unsafe` type. As a result, OpenJ9 crashes when executing the test case, whereas the other two JVMs, HotSpot and GraalVM, execute it successfully.

This test case is generated through two testing iterations. Specifically, the statement in line 11 is inserted in the first iteration, which is extracted from a historical test case (Copymemory.java 2017). The generated test case is returned to the seed pool due to trigger-

Table 2 SSFuzz-exposed bugs

JVMs	Submitted	Confirmed	Fixed	Undetermined
HotSpot	9	5	1	4
OpenJ9	8	7	2	1
GraalVM	3	2	0	1
Total	20	14	3	6

ing an anomalous behavior. In the next iteration, the test case is selected again and assembled with the code segment in lines 18-20, which is extracted from another historical test case (Test7190310_unsafe.java 2017). The broken dependencies, including variable `len`, `addr`, and `value`, are fixed during test case synthesis (lines 13-16).

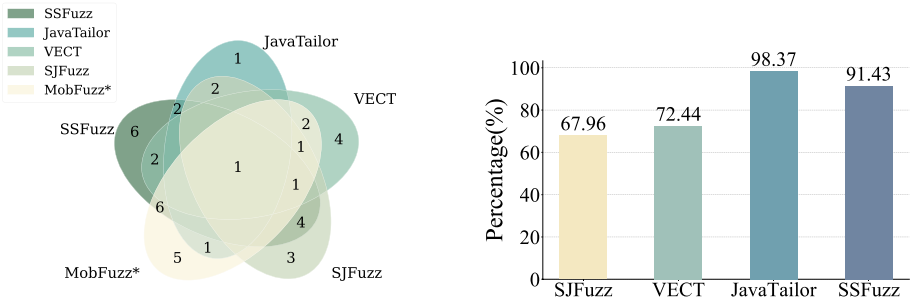
5.2 Compare to Prior Fuzzers (RQ2)

In this experiment, we compare SSFuzz to the four baselines outlined in Section 4.3 using the metrics described in Section 4.4. Each tool is allocated a budget of 24 hours of concurrent test runs, including the time required for test case generation. To ensure a fair comparison, each method is run independently on our evaluation platform.

Bug-triggering Capability The Venn diagram in Fig. 3a presents the overlapping and unique anomalies detected by each tool. SSFuzz identifies six unique bugs, and MobFuzz* finds five unique bugs, illustrating the effectiveness of bug-triggering code segments and synthesis methods in SSFuzz. VECT and SJFuzz contribute four and three unique cases, respectively, whereas JavaTailor identifies only one.

Moreover, SSFuzz exhibits extensive overlap with other tools, sharing six cases with MobFuzz*, four with SJFuzz, and two each with JavaTailor and VECT. This indicates that SSFuzz not only uncovers exclusive cases but also effectively covers a broad range of common cases detected by others.

Syntax Passing Rate Figure 3b shows the rate of test cases that can run `javac` successfully for each tool. JavaTailor gets the highest syntax passing rate, followed by SSFuzz. The slightly lower rate of SSFuzz is likely a trade-off of its methodology: it retains internal classes and methods during code segment extraction to maximize the diversity of the code segment corpus. While this decision enhances variety, it also introduces more complex dependencies that are challenging to fully resolve. The three synthesis strategies mitigate these issues; however, the results demonstrate that even LLM-based prompting cannot guarantee the syntactic correctness of all generated test cases. As a mutation-based fuzzer, SJFuzz gets the lowest syntax passing rates.



(a) Anomalies discovered by compared baselines and SSFuzz.

(b) Syntax passing rates across baselines and SSFuzz.

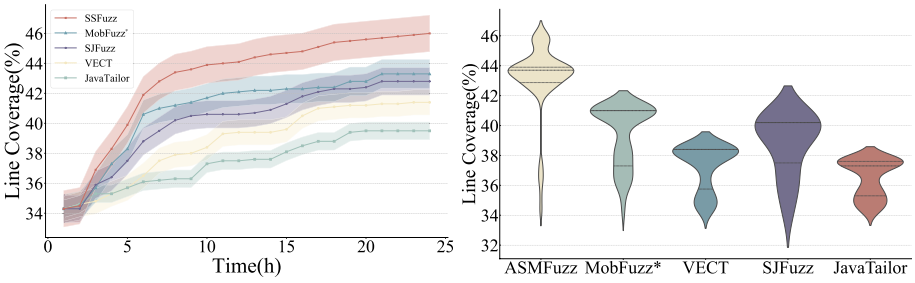
Fig. 3 Comparison results of bug detection ability and syntax passing rate

Code Coverage We employ two approaches to illustrate the changes in line coverage when different fuzzers are used to test JVMs. Figure 4a depicts the trend of line coverage achieved by various fuzzing tools over time. Due to the high-quality code segment corpus, the line coverage of SSFuzz increases rapidly from the start. Its multi-feedback-guided scheduler further allows it to attain the highest coverage without plateauing during the experiment. The growth speed of the line coverage of MobFuzz* is relatively lower than SSFuzz. This is primarily because the time consumed by metric statistics was not sufficiently considered when scheduling metric combinations, resulting in prolonged feedback time and consequently reducing coverage efficiency within a given testing period. SJFuzz integrates both the seed program scheduler and the mutator scheduler, effectively mitigating the limitation of single feedback. However, its schedulers do not consider the coverage information during the testing process, resulting in lower code coverage compared to SSFuzz and MobFuzz*. As an enhancement to JavaTailor, VECT achieves better line coverage by clustering code segments. However, the redundant clusters and the coarse-grained scheduler prevent it from achieving even higher coverage.

Figure 4b shows the distribution of line coverage across all test runs. For SSFuzz, higher coverage values occur more frequently, indicating that most test cases it generates are capable of exercising deep code branches of JVMs. In comparison, MobFuzz* performs less effectively. Since the scheduling module is the only difference between SSFuzz and MobFuzz*, these results confirm that the scheduling design of SSFuzz substantially enhances fuzzing efficiency.

Further Analysis Between JavaTailor and SSFuzz To further demonstrate the effectiveness of test case generation in SSFuzz, we compare the line code coverage between JavaTailor and SSFuzz across different JVM core modules. For a fair comparison, we use the same set of seed programs, disable the scheduler in SSFuzz, and run both tools on the same JVMs for 24 hours. The results are presented in Fig. 5. The modules are selected based on the coverage metrics and categorical labels in the PRs.

As shown in Fig. 5, SSFuzz achieves relatively higher function coverage across most modules. In particular, the function coverage of SSFuzz for the `GC`, `Prims`, and `JFR` modules (20.48%, 18.96%, 13.11%) is significantly higher than that of JavaTailor. Those three



(a) Line coverage trend across baselines and SSFuzz.

(b) Line coverage distribution across baselines and SSFuzz.

Fig. 4 Comparison results of line coverage

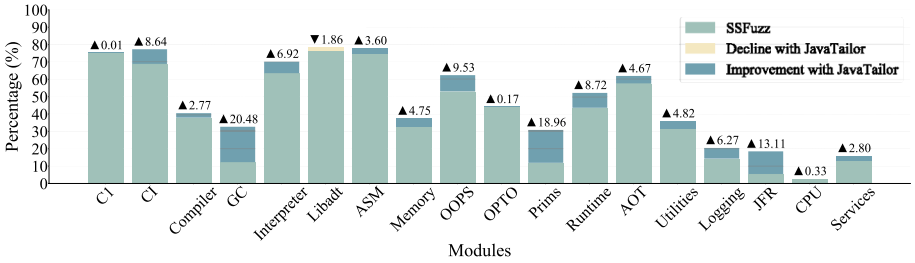


Fig. 5 Detailed comparison of line coverage between SSFuzz and JavaTailor

modules are related to Garbage Collection, primitive operations, and Java Flight Recorder, all of which are frequently updated. The only exception is the `Libadt` module, where JavaTailor demonstrates a 1.86% improvement over SSFuzz. However, `Libadt` has seen limited modifications in recent years, with only 15 commits since 2020 (Commit record of module libadt 2020) and thus is not a current focus of testing. These results demonstrate that the test case generation strategy of SSFuzz enables deeper exploration of execution paths in frequently updated modules.

5.3 Effectiveness of Synthesizing Method (RQ3)

To investigate the contribution of different ways of synthesizing test cases, we implement four variants of SSFuzz: (1) SSFuzz-A, which uses seed programs as test cases, (2) SSFuzz-B, which uses variables declared in seed programs, (3) SSFuzz-C, which searches variables stored in the variable declaration database $\mathcal{V}$, (4) SSFuzz-D, which generates initialization statements using an LLM. Figure 6 shows the anomalous behavior and code coverage of four variants and SSFuzz after a 24-hour run.

Overall, SSFuzz achieves the highest values across all three types of code coverage and exposes the most anomalous behaviors, highlighting the effectiveness of its combined

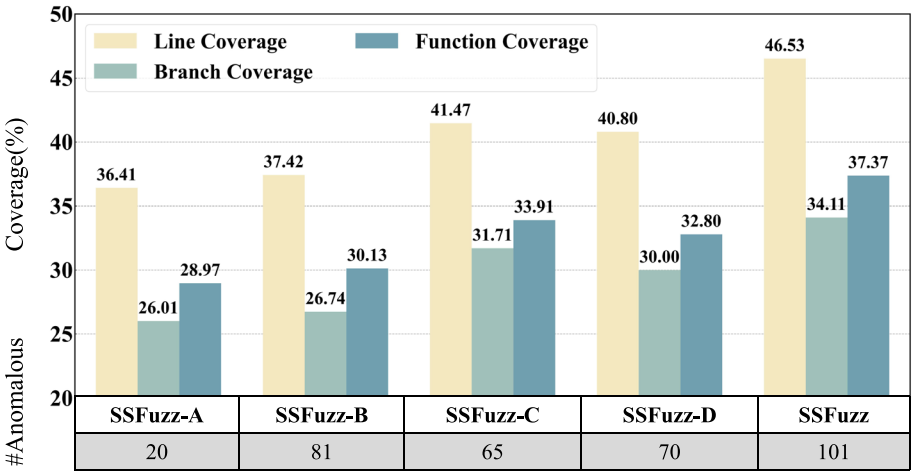


Fig. 6 Code coverage and anomalous behaviors of four synthesis-based variants and SSFuzz

approach to synthesizing test cases. In contrast, SSFuzz-A exhibits the lowest coverage and fewest detected anomalies, emphasizing the necessity of incorporating synthesis methods. Compared with SSFuzz-A, SSFuzz-B strengthens the interaction between code segments and seed programs, thereby improving both coverage and anomaly detection. Meanwhile, SSFuzz-C and SSFuzz-D yield comparable levels of coverage and anomaly detection, confirming their effectiveness.

To further evaluate the effectiveness of the three synthesis methods, we present the results of variable completion under different configurations. Specifically, we select 6,000 pairs of code segments and seed programs, apply different combinations of the three methods, and record the number of test cases that successfully complete all missing variables.

Figure 7 reports the detailed results. Method1 reuses variables declared in seed programs, Method2 retrieves variables from the variable declaration database $\mathcal{V}$, and Method3 generates initialization statements using an LLM. The call sequence for any combination of two methods follows Algorithm 1. The x-axis denotes the number of missing variable declarations before synthesis, while the y-axis denotes the number of test cases that complete all missing declarations after synthesis. The depth of the bar color represents the number of complete cases; darker shades signify a higher number of test cases with complete variables.

The completion effect achieved by Method1 is relatively limited. In contrast, when using Method2 or Method3, the effectiveness is significantly improved. The three synthesis methods employed in this study operate collaboratively, thereby attaining a higher level of completion performance. The few remaining missing variables are typically attributed to the complexity of the absent variable types, which often require the integration of multiple

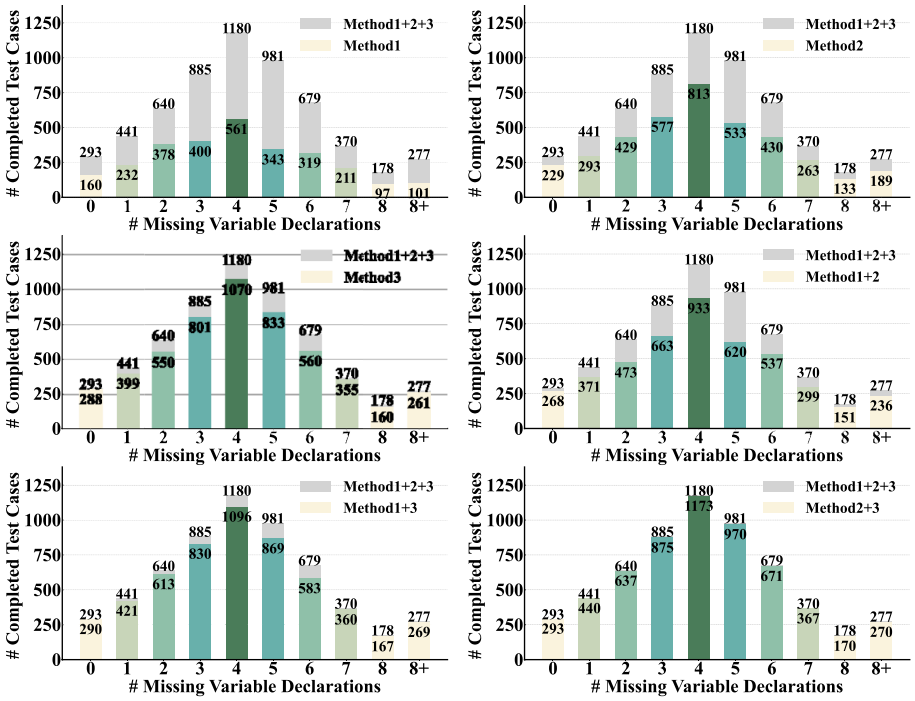


Fig. 7 Variable completion results under different synthesis configurations

undefined variables, or to the inability of the LLM to accurately generate the specific logical content of the missing classes during the synthesis process.

5.4 Effectiveness of Multi-Feedback Guided Scheduling (RQ3)

To evaluate the contribution of two schedulers, we design two variants: SSFuzz-E, which solely activates the seed program scheduler, and SSFuzz-F, which only activates the code segment scheduler. Then, we run two variants and SSFuzz for 24 hours.

As illustrated in Fig. 8, the difference between SSFuzz-E and SSFuzz-F is minimal. Specifically, SSFuzz-E detects more bugs, whereas SSFuzz-F achieves higher code coverage. Overall, SSFuzz provides the highest coverage and detects the most anomalies. These findings suggest that combining the two schedulers is more effective than using each one independently.

5.5 Selection of LLMs (RQ4)

To clarify the influence of using different LLMs in test case synthesis, we select eight widely used LLMs and generate 600 test cases with each model. Table 3 presents the generation time, syntax passing rate of generated test cases, and line coverage of HotSpot. The highest values for each metric are presented in bold.

Claude-3-5-Sonnet achieved the highest line coverage, and Claude-3-7-Sonnet achieved the highest syntax passing rate. This is due to Claude's specialized optimizations in code syntax tree modeling and program state inference. However, this comes at the cost of a long runtime. In contrast, OpenAI's GPT-4-Turbo strikes a balance between quality and efficiency, surpassing its predecessor, GPT-3.5-Turbo, with double the syntax passing rate and only a minor difference in coverage. Notably, its generation time is 0.5 times that of GPT-3.5-Turbo. The two DeepSeek models illustrate how model design goals significantly influence performance characteristics. The general-purpose DeepSeek-V3, optimized for efficiency with multi-token prediction, achieves a 62.70% syntax passing rate in 19.09

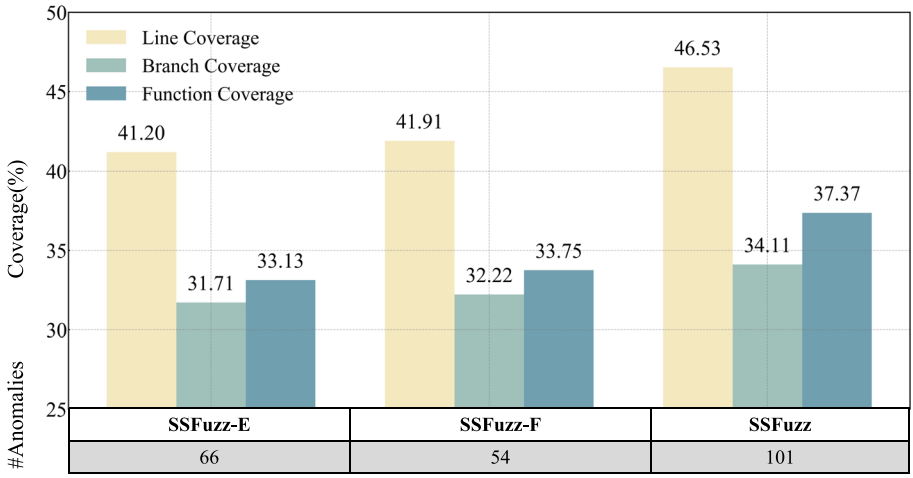


Fig. 8 Code coverage and anomalous behaviors of four scheduler-based variants and SSFuzz

Table 3 The quality of generated test cases using different models

Model	TT(s)	AT(s)	SPR(%)	LC(%)
GPT-3.5-Turbo (OpenAI 2023a)	4689.47	7.82	28.94	40.3
GPT-4-Turbo (OpenAI 2023b)	**2550.37**⋆	**4.25**⋆	59.93	40.9
DeepSeek-R1 (DeepSeek 2025)	61087.51	101.81	66.82	43.6
DeepSeek-V3 (DeepSeek 2024)	11453.29	19.09	62.70	40.8
Llama-3.1-405b-Instruct (Meta 2024)	7812.11	13.02	58.24	39.1
Claude-3-5-Sonnet-20240620 (Anthropic 2024)	14844.89	24.74	71.81	**45.4**⋆
Claude-3-7-Sonnet-20250219 (Anthropic 2025)	12262.41	20.44	**81.44**⋆	42.2
Grok-3 (x.ai 2025)	18761.29	31.27	75.90	45.0

TT: **T**otal **T**ime, **AT**: **A**verage **T**ime, **SPR**: **S**yntax **P**assing **R**ate, **LC**: **L**ine **C**overage

seconds per case. In contrast, the reinforcement learning-based DeepSeek-R1, which achieves the highest coverage at 43.6%, suffers a substantial increase in runtime to 101.81 seconds per case due to its complex reasoning process.

6 Related Work

A primary challenge in compiler fuzzing is generating diverse and valid programs as test cases. Existing compiler fuzzers typically employ generative or mutational techniques to generate test cases and use schedulers to guide the selection of seed programs, mutators, or mutated test cases.

Generative Compiler Fuzzing Generation-based fuzzers generate test cases from stochastic grammar rules (Yang et al. 2011; Liu et al. 2023), carefully designed templates (funfuzz 2023; Zhang et al. 2017; Hua and Khurshid 2017; Zang et al. 2023), or generative models (Ye et al. 2023, 2021; Lee et al. 2020; Yang et al. 2024; Xia et al. 2024). One of the representative works using grammar rules is jsfunfuzz (funfuzz 2023). It designs several context-free grammars to generate JS test cases. To improve syntax correctness, researchers have proposed template-based compiler fuzzing. SPE (Zhang et al. 2017) focuses on creating all possible programs from a given structure and set of variables and presents a novel algorithm that reduces the complexity of finding unique programs. Using JAttack (Zang et al. 2023), developers can design templates with holes, thereby constraining the exploration of test input into a syntax-correct range. With the rapid advancement of generative models, modern compiler fuzzers are increasingly integrated with powerful models. COMFORT (Ye et al. 2021) and COMFUZZ (Ye et al. 2023) fine-tune transformer-based models to generate diverse and syntactically correct test cases. COMFORT utilizes JavaScript programs sourced from GitHub repositories as its fine-tuning dataset. COMFUZZ incorporates historical test programs into its fine-tuning dataset to enhance the bug-triggering potential of the generated test

cases. Recently, LLMs have been used to assist compiler testing. WhiteFox (Yang et al. 2024) utilizes a multi-agent framework to automatically infer testing requirements from compiler source code and generate high-level test programs aimed at triggering optimizations.

Mutational Compiler Fuzzing Mutation-based fuzzers generate diverse test cases by inserting, deleting, or modifying seed programs (Zhao et al. 2022; Gao et al. 2023; Chen et al. 2016, 2019; Han et al. 2019; Park et al. 2020; Wu et al. 2023; Wang et al. 2023; Jia et al. 2023; Xu et al. 2024). ClassFuzz (Chen et al. 2016) designs a set of predefined mutators to mutate seed programs on the IR level (Jimple) and employs Markov Chain Monte Carlo (MCMC) sampling to guide mutator selection. However, ClassFuzz mainly produces illegal bytecode files and uncovers bugs in JVMs' startup processes. To solve this limitation, ClassMing (Chen et al. 2019) inserts instructions such as goto, return, throw, etc., into the bytecode file, altering its control flow and data flow. JavaTailor (Zhao et al. 2022) is the first history-driven test program synthesis technique. It inserts code segments extracted from historical test programs into seed programs by a random strategy. To decrease the large space of code segments, VECT clusters semantically similar code segments and priority clusters based on a scheduler. As the number of JIT-related bugs increases, more researchers are beginning to focus on JIT testing. These works (Wu et al. 2023; Wang et al. 2023; Jia et al. 2023) design customized mutators that can frequently activate JIT.

Unlike previous works, SSFuzz focuses on collecting code segments with high bug-triggering potential and developing effective strategies for synthesizing code segments and seed programs. Additionally, the seed program and code segment scheduler ensure continuous testing of error-prone modules.

7 Threats to Validity

Threats to Internal Validity In Section 5.2, we compare SSFuzz with JavaTailor using both their respective seed programs and the seed programs of SSFuzz. Although we aimed to conduct the comparison under identical code segment sets to isolate the effects of different code segment sources and synthesis strategies, the comparison remains limited by differences in their code manipulation levels. JavaTailor relies on Soot for bytecode-level manipulation, which limits it to code segments that can be successfully compiled. In contrast, SSFuzz can synthesize incomplete code segments into executable test cases by leveraging LLM-based code completion. We recognize this discrepancy as a potential threat to internal validity and plan to address it in future work.

8 Discussion

Generality Our work takes the JVM as a case study, but the core concept of synthesizing and scheduling bug-triggering code segments can be applied to testing other compilers as well. The necessary modifications involve replacing the code parsing and coverage counting modules. The remaining components are generalizable.

Range of PR Collection The PRs collected for this experiment span from March 2022 to February 2024. Previous research (Zhu and Böhme 2021) suggests that frequently updated modules are more likely to conceal novel bugs. In future studies, we plan to expand the PR collection to include test programs from the past one, two, or more years to assess the impact of this broader timeframe.

9 Conclusion

We propose SSFuzz, a new history-driven compiler testing technique. It synthesizes bug-triggering code segments and employs multi-feedback guided scheduling to uncover novel bugs. SSFuzz collects test programs from PRs and extracts bug-triggering code segments from those programs. To conduct effective testing, SSFuzz employs two schedulers: one for seed programs and another for code segments. We implemented a prototype for SSFuzz and conducted an extensive experiment. During the issue-submission period, SSFuzz identified 20 previously unknown bugs, 14 of which have been confirmed by developers. The overall results demonstrated that SSFuzz can detect uncovered bugs and enable comprehensive testing on key modules.

Author Contributions Tianmin Hu (first author, co-first author) and Guixin Ye (fourth author, corresponding author) contributed to the conceptualization, methodology, and writing of the paper, including drafting and revising the manuscript. Zhenye Fan (second author, co-first author) was responsible for system implementation and conducted all experiments. Zhanbo Ye (third author) created all the figures and tables and participated in several experiments. Lu He (fifth author) contributed to certain aspects of system implementation and experimentation. All authors reviewed and approved the final manuscript.

Funding This work was supported in part by the National Natural Science Foundation of China (NSFC) under grant agreements (62472351), the China Postdoctoral Science Foundation Fellowship (2022M712575).

Data Availability The data associated with this work are publicly available on the following website: https://github.com/ssfuzz/ssfuzz

Code Availability The code associated with this work is publicly available on the following website: https://github.com/ssfuzz/ssfuzz

Declarations

Ethical Approval This study did not require ethics approval.

Consent to Participate Not applicable.

Consent for Publication Not applicable.

Competing Interests The authors have no relevant financial or non-financial interests to disclose.

References

"Azulzulu," https://www.azul.com/products/core/. Accessed February 2025
"Azuzing," https://www.azul.com/products/prime/. Accessed February 2025

"Commit record of module libadt," https://github.com/openjdk/jdk/commits/master/src/hotspot/share/libadt. Accessed February 2025
"Copymemory.java," https://github.com/openjdk/jdk11u-dev/blob/70862e011ba642a782cf28004d2f9dd807971788/test/jdk/sun/misc/CopyMemory.java#L66C1-L68C10
"funfuzz," https://github.com/MozillaSecurity/funfuzz. Accessed February 2025
"Gcov," https://gcc.gnu.org/onlinedocs/gcc/Gcov.html. Accessed February 2025
"Graalvm," https://www.graalvm.org/. Accessed February 2025
"Hotspot," https://openjdk.org/groups/hotspot/. Accessed February 2025
"Jikesrvm," https://github.com/JikesRVM/JikesRVM/. Accessed February 2025
"Openj9," https://www.eclipse.org/openj9/. Accessed February 2025
"Pr#13855," https://github.com/openjdk/jdk/pull/13855. Accessed February 2025
"Pr#17699," https://github.com/eclipse-openj9/openj9/pull/17699/files
"Test7190310_unsafe.java," https://github.com/openjdk/jdk11u-dev/blob/70862e011ba642a782cf28004d2f9dd807971788/test/hotspot/jtreg/compiler/c2/Test7190310_unsafe.java#L61C1-L62C1
Anthropic (2024) "Claude-3.5," Anthropic, Technical Report. https://assets.anthropic.com/m/61e7d27f8c8f5919/original/Claude-3-Model-Card.pdf
Anthropic (2025) "Claude 3.7," Anthropic, Technical Report. https://assets.anthropic.com/m/785e231869ea8b3b/original/claude-3-7-sonnet-system-card.pdf
Böhme M, Manès VJ, Cha SK (2020) Boosting fuzzer efficiency: an information theoretic perspective. In: Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering. pp 678–689
Chen Y, . Su T, Su Z (2019) Deep differential testing of JVM implementations. In: 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE). IEEE, pp 1257–1268
Chen J, Patra J, Pradel M, Xiong Y, Zhang H, Hao D, Zhang L (2020) A survey of compiler testing. ACM Comput Surv (CSUR) 53(1):1–36
Chen Y, Su T, Sun C, Su Z, Zhao J (2016) Coverage-directed differential testing of JVM implementations. In: Proceedings of the 37th ACM SIGPLAN conference on programming language design and implementation. pp 85–99
DeepSeek (2024) "Deepseek-v3," DeepSeek, Technical Report. https://github.com/deepseek-ai/DeepSeek-V3/blob/main/DeepSeek_V3.pdf
DeepSeek (2025) "Deepseek-r1," DeepSeek, Technical Report. https://github.com/deepseek-ai/DeepSeek-R1/blob/main/DeepSeek_R1.pdf
Gao T, Chen J, Zhao Y, Zhang Y, Zhang L (2023) Vectorizing program ingredients for better JVM testing. In: Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis. pp 526–537
Han H, Oh D, Cha SK (2019) CodeAlchemist: semantics-aware code generation to find vulnerabilities in Javascript engines. In: Proceedings of the 26th annual network and distributed system security symposium (NDSS)
Hinds N, Larson P, Franke H, Ridgeway M (2014) Using code coverage tools in the Linux kernel. ACM SIGSOFT Software Engineering Notes (ACM'04). pp 70–81
Holler C, Herzig K, Zeller A (2012) Fuzzing with code fragments. In: Proceedings of the 21st USENIX security symposium (USENIX Security). pp 445–458
Hosseini R, Brusilovsky P (2013) Javaparser: a fine-grain concept indexing tool for Java problems. In: CEUR workshop proceedings, vol 1009. University of Pittsburgh, pp 60–63
Hua J, Khurshid S (2017) EdSketch: Execution-driven sketching for java. In: Proceedings of the 24th ACM SIGSOFT International SPIN symposium on model checking of software. pp 162–171
Huang D, Tristan J-B, Morrisett G (2017) Compiling Markov Chain Monte Carlo algorithms for probabilistic modeling. In: Proceedings of the 38th ACM SIGPLAN conference on programming language design and implementation, ser. PLDI 2017. New York, NY, USA: Association for Computing Machinery, pp 111–125. https://doi.org/10.1145/3062341.3062375
Jia H, Wen M, Xie Z, Guo X, Wu R, Sun M, Chen K, Jin H (2023) Detecting JVM JIT compiler bugs via exploring two-dimensional input spaces. In: 2023 IEEE/ACM 45th international conference on software engineering (ICSE). pp 43–55
Lee S, Han H, Cha SK, Son S (2020) Montage: A neural network language model-guided Javascript fuzzer. In: Proceedings of the 29th USENIX security symposium (USENIX Security). USENIX, pp 2613–2630
Liu J, Lin J, Ruffy F, Tan C, Li J, Panda A, Zhang L (2023) NNSmith: generating diverse and valid test cases for deep learning compilers. In: Proceedings of the 28th ACM international conference on architectural support for programming languages and operating systems, vol 2, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, pp 530–543. https://doi.org/10.1145/3575693.3575707

Liu B, Meng G, Zou W, Gong Q, Li F, Lin M, Sun D, Huo W, Zhang C (2020) A large-scale empirical study on vulnerability distribution within projects and the lessons learned. In: 2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE). IEEE, pp 1547–1559

Meta (2024) "Llama 3.1," Meta, Technical Report. https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md

OpenAI (2023) "Gpt-4," OpenAI, Technical Report. https://openai.com/index/gpt-4-research/

OpenAI (2023) Gpt-3.5 turbo, OpenAI, Technical Report. https://openai.com/index/gpt-3-5-turbo-fine-tuning-and-api-updates/

Park S, Xu W, Yun I, Jang D, Kim T (2020) Fuzzing Javascript engines with aspect-preserving mutation. In: Proceedings of the 41st IEEE symposium on security and privacy (S&P). pp 1629–1642

Shannon CE (1948) A mathematical theory of communication. The Bell Syst Technical J 27(3):379–423

Wang J, Song C, Yin H (2021) Reinforcement learning-based hierarchical seed scheduling for greybox fuzzing

Wang J, Zhang Z, Liu S, Du X, Chen J (2023) FuzzJIT: Oracle-Enhanced fuzzing for JavaScript engine JIT compiler. In: 32nd USENIX security symposium (USENIX Security 23). Anaheim, CA: USENIX Association. pp 1865–1882. https://www.usenix.org/conference/usenixsecurity23/presentation/wang-junjie

Wu M, Lu M, Cui H, Chen J, Zhang Y, Zhang L (2023) JITfuzz: Coverage-guided fuzzing for JVM just-in-time compilers. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, pp 56–68

Wu M, Ouyang Y, Lu M, Chen J, Zhao Y, Cui H, Guo Y, Zhang Y (2023) SJFuzz: seed & mutator scheduling for JVM fuzzing. In: 2023 The ACM joint European software engineering conference and symposium on the foundations of software engineering (ESEC/FSE)

x.ai (2025) "Grok 3 beta," x.ai, Technical Report. https://x.ai/blog/grok-3

Xia CS, Paltenghi M, Le Tian J, Pradel M, Zhang L (2024) Fuzz4all: universal fuzzing with large language models. In: Proceedings of the IEEE/ACM 46th international conference on software engineering, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3597503.3639121

Xu H, Jiang Z, Wang Y, Fan S, Xu S, Xie P, Fu S, Payer M (2024) Fuzzing Javascript engines with a graph-based IR. In: Proceedings of the 2024 on ACM SIGSAC conference on computer and communications security, ser. CCS '24. New York, NY, USA: Association for Computing Machinery, pp 3734–3748. https://doi.org/10.1145/3658644.3690336

Yang X, Chen Y, Eide E, Regehr J (2011) Finding and understanding bugs in C compilers. In: Proceedings of the 32nd ACM SIGPLAN conference on programming language design and implementation, ser. PLDI '11. New York, NY, USA: Association for Computing Machinery, pp 283–294. https://doi.org/10.1145/1993498.1993532

Yang C, Deng Y, Lu R, Yao J, Liu J, Jabbarvand R, Zhang L (2024) Whitefox: white-box compiler fuzzing empowered by large language models. Proc ACM Program Lang 8(OOPSLA2). https://doi.org/10.1145/3689736

Ye G, Hu T, Tang Z, Fan Z, Tan SH, Zhang B, Qian W, Wang Z (2023) A generative and mutational approach for synthesizing bug-exposing test cases to guide compiler fuzzing. In: Proceedings of the 31st ACM joint European software engineering conference and symposium on the foundations of software engineering. Association for Computing Machinery (ACM)

Ye G, Tang Z, Tan SH, Huang S, Fang D, Sun X, Bian L, Wang H, Wang Z (2021) Automated conformance testing for Javascript engines via deep compiler fuzzing. In: Proceedings of the 42nd ACM SIGPLAN international conference on programming language design and implementation. pp 435–450

Zang Z, Wiatrek N, Gligoric M, Shi A (2023) Compiler testing using template Java programs. In: Proceedings of the 37th IEEE/ACM international conference on automated software engineering, ser. ASE '22. New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3551349.3556958

Zhang Q, Sun C, Su Z (2017) Skeletal program enumeration for rigorous compiler testing. In: Proceedings of the 38th ACM SIGPLAN conference on programming language design and implementation, ser. PLDI 2017. New York, NY, USA: Association for Computing Machinery, pp 347–361. https://doi.org/10.1145/3062341.3062379

Zhang G, Wang P, Yue T, Kong X, Huang S, Zhou X, Lu K (2024) MobFuzz: adaptive multi-objective optimization in gray-box fuzzing. arXiv:2401.15956

Zhao Y, Wang Z, Chen J, Liu M, Wu M, Zhang Y, Zhang L (2022) History-driven test program synthesis for JVM testing. In: 2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE). IEEE, pp 1133–1144

Zhong H (2022) Enriching compiler testing with real program from bug report. In: Proceedings of the 37th IEEE/ACM international conference on automated software engineering. pp 1–12

Zhu X, Böhme M (2021) Regression greybox fuzzing. In: Proceedings of the 2021 ACM SIGSAC conference on computer and communications security, ser. CCS '21. New York, NY, USA: Association for Computing Machinery, pp 2169–2182. https://doi.org/10.1145/3460120.3484596

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Tianmin Hu received the B.S. degree in Computer Science and Technology from Northwest University, Xi'an, China, in 2021. She is currently a combined Master-PhD student in Software Engineering at the School of Computer Science (School of Software) at Northwest University. Her research interests include software testing and compilers.



Zhenye Fan pursued his master's degree at Northwest University, Xi'an, China, and is currently a Ph.D. candidate at the National University of Defense Technology, Changsha, China. He specializes in research on fuzzing, AI security, and software system security.



Zhanbo Ye received the B.S. degree in Information Science and Technology from Northwest University, Xi'an, China, in 2025. His research interests include LLM security.



Guixin Ye is an Associate Professor at the School of Computer Science, Northwest University. He received his Ph.D. degree from Northwest University, China, in 2019. His research interests include software security, software testing, authentication, and privacy. He has published more than 30 papers in international peer-reviewed conferences and transactions, including PLDI, CCS, NDSS, FSE, ISSTA, TIFS, and TDSC.



Lu He is a lecturer at the School of Computer Science, Northwest University. He received his Ph.D. degree from Xi'an Jiao Tong University, China, in 2013. His research interests include software security, AI security network, and information security. He has published more than 20 papers in internal and international peer-reviewed conferences and transactions.